

Integration Test Case Document

=====

Project: Ezra Booking System
Module: Medical Questionnaire & Privacy
Test Case ID: TC-EZRA-MED-PRIV-001
Prepared By: Denny Nguyen
Date: 2026-03-14
Version: 1.0

=====

1. Objective

Verify that when a member begins a medical questionnaire, they can only access their own medical data, and all attempts to access other members' data are blocked, enforcing privacy and data security.

2. Scope

In-Scope: - Member login and authentication - Begin Medical Questionnaire flow - Questionnaire data retrieval and display - Cross-member access attempts (UI, URL, API)

Out-of-Scope: - Data entry correctness within the questionnaire - Non-medical booking workflows

3. Preconditions

1. Test environment mirrors production configuration.
2. Test accounts available:
 - Member A: Has medical data and questionnaire in progress.
 - Member B: Has medical data and questionnaire in progress.
3. Authentication and API tokens are properly configured.

4. Test Data

User	Record Type	Example Data
Member A	Medical Questionnaire	Blood Pressure, Allergies
Member B	Medical Questionnaire	Diabetes History, Medications

5. Test Steps

5.1 Start Questionnaire (Authorized Access)

- Log in as Member A.
- Navigate to Begin Medical Questionnaire.
- Verify that only Member A's data is displayed. **Expected Result:** Member A can access only their own medical data.

5.2 Attempt Unauthorized Access (UI)

- While logged in as Member A, manually try to access Member B's questionnaire URL.
- Observe system behavior. **Expected Result:** Access denied; error message displayed (e.g., "Unauthorized Access").

5.3 Attempt Unauthorized Access (API)

- Using Member A's token, call the API endpoint for Member B's questionnaire.
- Record API response. **Expected Result:** HTTP 403 Forbidden; no data returned.

5.4 Cross-Session Attempt

- Log in as Member B in another browser/session.
- Attempt to access Member A's questionnaire using URL, API, or session tricks. **Expected Result:** Access denied; session isolation enforced.

5.5 Data Leakage Check

- Export, print, or share the questionnaire as Member A.
- Verify no information from Member B is included. **Expected Result:** Only Member A's medical data is accessible; no cross-member leakage.

6. Expected Results

- Only the logged-in member can access their own medical questionnaire.
- Unauthorized access attempts are blocked.
- Proper HTTP status codes and error messages are returned.
- Audit logs record all failed access attempts.

7. Pass/Fail Criteria

- **Pass:** No member can access another's medical questionnaire.
- **Fail:** Any member is able to view or retrieve another member's questionnaire data.

8. Dependencies

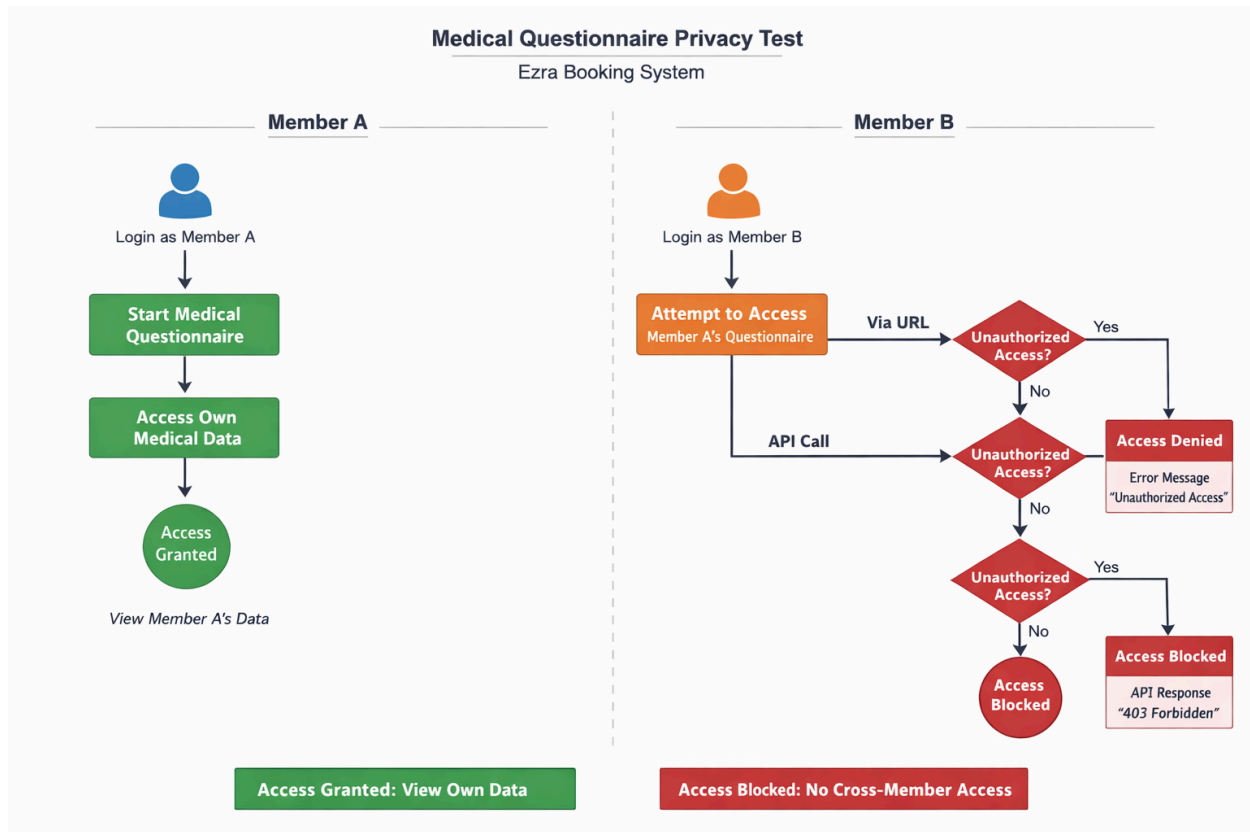
- Authentication & authorization services fully functional.
- RBAC enforced at front-end, back-end, and API layers.

- Secure session handling enabled.

9. Notes

- Include this test case in regression testing whenever the medical questionnaire, authentication, or access control logic is updated.
- Ensure the test environment contains production-like security configurations.

10. Workflow Diagram



11. Advanced Critical Scenarios

11.1 Session Hijacking Simulation

- Attempt to use a stolen session token from Member A to access Member B's questionnaire. **Expected Result:** Stolen tokens are invalidated; forced logout occurs; attempts logged.

11.2 URL/ID Manipulation

- Manually alter questionnaire IDs or URLs to access other members' data. **Expected Result:** Access denied; no data leakage.

11.3 Parameter Pollution / API Injection

- Send API requests with multiple conflicting member identifiers. **Expected Result:** Only authenticated user data returned; invalid parameters rejected.

11.4 Concurrent Access / Race Condition

- Two requests from different members simultaneously target the same data. **Expected Result:** No cross-access occurs; each sees only their own data.

11.5 Cached Data / Browser Storage Leakage

- Log out Member A, log in Member B on the same browser. **Expected Result:** Previous session data cleared; Member B cannot see Member A's questionnaire.

11.6 Copy-Paste / Export Abuse

- Attempt to import/export data across members via UI or API. **Expected Result:** Ownership validation blocks cross-member import.

11.7 Edge Case Data Injection

- Insert long strings, special characters, or JSON/XML payloads in questionnaire fields. **Expected Result:** Input validation blocks malicious payloads; no data leakage.

11.8 Multi-Role Escalation Test

- Staff/admin account tries to access member questionnaires beyond permission. **Expected Result:** Least privilege enforced; attempts logged.

11.9 API Rate-Limit & Timing Attack

- Flood API with rapid queries attempting different member IDs. **Expected Result:** Rate-limiting triggers; no data leakage.

11.10 Third-Party Integration Abuse

- External systems attempt to fetch/inject data for another member. **Expected Result:** Only authorized data flows; cross-member access impossible.

Part 2 – Integration Test Using HTTP Requests

Step 1 – Authenticate Member A

Purpose: Verify that Member A can authenticate and obtain a valid token.

HTTP Request:

POST /api/v1/auth/login

Host: api.ezra.com

Content-Type: application/json

```
{  
  "email": "memberA@test.com",  
  "password": "Password123"  
}
```

Expected Response: HTTP 200 OK with authentication token.

Step 2 – Begin Medical Questionnaire

HTTP Request:

POST /api/v1/medical-questionnaire/start

Authorization: Bearer memberA_token

Content-Type: application/json

```
{  
  "member_id": "memberA_id"  
}
```

Expected Response: HTTP 200 OK. Questionnaire session created.

Step 3 – Retrieve Member A Questionnaire

HTTP Request:

GET /api/v1/medical-questionnaire/q12345

Authorization: Bearer memberA_token

Expected Response: HTTP 200 OK returning questionnaire data belonging to Member A.

Step 4 – Unauthorized Access Attempt

HTTP Request:

GET /api/v1/medical-questionnaire/q67890
Authorization: Bearer memberA_token

q67890 = Member B's questionnaire

Expected Response: HTTP 403 Forbidden. No medical data returned.

Step 5 – Parameter Manipulation

HTTP Request:

GET /api/v1/medical-questionnaire?member_id=memberB_id
Authorization: Bearer memberA_token

Expected Response: HTTP 403 Forbidden. API ignores client-supplied member ID.

Step 6 – Unauthorized Submission Attempt

HTTP Request:

POST /api/v1/medical-questionnaire/submit
Authorization: Bearer memberA_token
Content-Type: application/json

```
{  
  "member_id": "memberB_id",  
  "questionnaire_id": "q67890"  
}
```

Expected Response: HTTP 403 Forbidden. User cannot modify another member's questionnaire.

EXTRA - Advanced Security Integration Tests

1. IDOR Attack Test

Purpose: Prevent Insecure Direct Object Reference attacks.

HTTP Request:

GET /api/v1/medical-questionnaire/q67890
Authorization: Bearer memberA_token

Expected Response: HTTP 403 Forbidden.

2. Token Replay Attack

Purpose: Ensure stolen tokens cannot be reused across sessions or users.

HTTP Request:

GET /api/v1/medical-questionnaire/q67890
Authorization: Bearer memberA_token

Expected Response: HTTP 401 Unauthorized or HTTP 403 Forbidden.

3. Session Switching / Cached Data Leakage

Purpose: Prevent data leakage when switching sessions.

Scenario: Member A logs out, Member B logs in on the same browser and attempts to access Member A's questionnaire.

HTTP Request:

GET /api/v1/medical-questionnaire/q12345
Authorization: Bearer memberB_token

Expected Response: HTTP 403 Forbidden.

Audit Security Validation

Audit Logging Verification

All unauthorized access attempts must generate audit logs.

Example Log Entry:

Timestamp: 2026-03-11T10:45:32Z
User: memberA_id
Target Resource: questionnaire q67890
Result: ACCESS_DENIED

Enumeration Attack Detection

Purpose: Detect multiple unauthorized access attempts.

Example:

GET /api/v1/medical-questionnaire/q10001
GET /api/v1/medical-questionnaire/q10002
GET /api/v1/medical-questionnaire/q10003

Expected Result: HTTP 403 responses and security monitoring alert triggered.

Audit Trail Verification

Purpose: Ensure all successful accesses are logged.

Example:

User memberA_id accessed questionnaire q12345

Result: ACCESS_GRANTED

Conclusion

The integration tests above verify that the system enforces strict authorization controls at the API layer. Users can only access their own medical data, while unauthorized attempts are blocked and logged. These protections ensure patient privacy and support healthcare security compliance.

Part 3 – Managing Security Quality for 100+ Sensitive Endpoints (Senior QA Perspective)

1. Overall Strategy

In a **production environment**, managing the security quality of over 100 sensitive endpoints requires a **multi-layered, risk-based approach** that combines automation, monitoring, and manual oversight.

Key principles:

1. **Risk-based prioritization** – classify endpoints based on sensitivity and exposure (PHI, payment info, authentication, scheduling, logs).
 2. **Automation-first testing** – ensure every endpoint has repeatable, automated security checks in CI/CD pipelines.
 3. **Continuous monitoring** – detect anomalies or misuse in real-time.
 4. **Compliance alignment** – ensure all actions adhere to HIPAA and internal security policies.
-

2. Production-Level Implementation Plan

a) Endpoint Classification & Risk Assessment

- **High-risk endpoints:** access/modify medical records, authentication, payment information.
- **Medium-risk endpoints:** user preferences, scheduling, notifications.
- **Low-risk endpoints:** public resources, informational endpoints.

Action: Assign testing frequency and monitoring intensity based on risk category.

b) Automated Security Testing in Production

- **Integration & API Security Tests**
 - Authorization enforcement (RBAC, multi-tenant isolation)
 - IDOR checks (ensuring users cannot access others' data)
 - Input validation for all query/body parameters (prevent injection attacks)
 - Token/session validation (including expiration and rotation)

- **Contract Testing**
 - Verify request/response schemas to prevent unexpected data leakage
- **Regression Testing**
 - Ensure no new deployments break security controls

Tooling: CI/CD pipelines should include automated security test suites for **all sensitive endpoints**, with results stored and reported.

c) Real-Time Monitoring and Logging

- **Audit logs** for all access attempts (success/failure) with:
 - User ID, endpoint, timestamp, IP, action, result
 - **Anomaly detection** for suspicious activity:
 - Repeated failed requests
 - Access from unusual IPs or geolocations
 - Rapid enumeration attempts
 - **Alerting:**
 - Security operations team receives notifications when thresholds are exceeded
 - Integration with SIEM for incident response
-

d) Manual Security Validation

- **Periodic penetration testing** of all endpoints
- **Edge-case validation** for multi-tenant isolation and session handling
- **Cross-environment testing:**

- Verify production-like behavior in staging before deployment
 - Simulate real-world attacks (IDOR, token replay, session hijacking)
-

3. Tradeoffs

Approach	Pros	Cons / Tradeoffs
Automated security testing	Scalable, repeatable, fast feedback	Requires maintenance; risk of false positives
Manual penetration testing	In-depth, detects unknown vulnerabilities	Time-consuming, not continuous
Continuous monitoring & logging	Real-time detection & compliance	Requires tuning, can generate alert fatigue
Risk-based prioritization	Focuses resources on critical endpoints	Lower-priority endpoints may lag in testing

4. Potential Risks

1. **Endpoint drift:** endpoints may change, breaking automated tests if not updated promptly.
2. **False negatives:** automation may miss subtle security gaps.
3. **Alert fatigue:** too many low-priority alerts may cause team to ignore real threats.
4. **Production testing challenges:** testing in production must avoid data corruption or privacy violations.

Mitigation: Use production-like staging environments, tokenized/mock data, and careful monitoring for live tests.

5. Recommended Best Practices

1. **Combine automation + manual checks** to maximize coverage.
 2. **Maintain endpoint documentation** with ownership, risk rating, and test coverage.
 3. **Centralize security governance** to ensure consistent access control policies.
 4. **Continuous risk assessment**: reevaluate endpoints after feature changes or external vulnerabilities.
 5. **Incident response readiness**: all logs and alerts feed into an actionable security response process.
-

6. Summary

Managing **100+ sensitive endpoints in production** is feasible by applying a **risk-based, automated, and monitored approach**:

- Prioritize endpoints by sensitivity.
- Apply automated integration and security tests in CI/CD.
- Maintain audit logs and real-time monitoring.
- Perform regular manual penetration tests.
- Continuously evaluate tradeoffs and risks.

This ensures **secure, compliant, and reliable endpoint management** while balancing operational efficiency and privacy protection.